

Using Large Language Models for System Modeling: Current State and Future Implications

1st Carina Freseemann

*Chair of Industrial Information Technology
Technical University of Berlin
Berlin, Germany
carina.freseemann@tu-berlin.de*

2nd Claudius Ellsel

*Chair of Industrial Information Technology
Technical University of Berlin
Berlin, Germany
ellsel@tu-berlin.de*

3rd Carl-Philipp Grunenwald

*Chair of Industrial Information Technology
Technical University of Berlin
Berlin, Germany
grunenwald@tu-berlin.de*

4th Rainer Stark

*Chair of Industrial Information Technology
Technical University of Berlin
Berlin, Germany
rainer.stark@tu-berlin.de*

Abstract—Model-Based Systems Engineering (MBSE) supports the efficient development of complex technical systems but requires significant expertise and manual effort. This paper investigates the current state of applying Large Language Models (LLMs) to support system modeling tasks, particularly in dealing with semi-formal languages such as the System Modeling Language (SysML). A structured literature review identifies twenty publications where LLMs are used for system modeling tasks, including the description of the technical set-up, evaluation of the solution and current existing gaps and limitations. Based on the findings, the paper proposes a modular prompting framework for translating SysML v1 to v2 structure diagrams, addressing key syntactic and semantic challenges. The results indicate potential for increased efficiency but highlight the need for evaluation metrics allowing for estimable LLM outputs. Finally, the paper outlines further research directions addressing sufficient and relevant context information as well as evaluation approaches.

Index Terms—large language models, MBSE, SysML, transformation, evaluation, MCP

I. INTRODUCTION

Industry and research target to shorten development cycles and at the same time to increase engineering efficiency and its innovation level [1]. Model-Based Systems Engineering (MBSE) is considered as one measure to manage complexity of technical systems across engineering disciplines and challenging demands on quality, reliability or safety [2]. It thereby contributes to efficient engineering processes. However, the effort to apply MBSE and to train specialized systems engineers remains a challenge for industry [3]. MBSE bases on the three pillars method, tool, and modeling language used for the description of the system models. Creating and maintaining these models is time consuming, and prone to errors [4], needs semantic understanding, as well as knowledge of the application of the modeling language [5]. Large language models (LLMs), as one special form of AI, are expected to support efficiency and creativity gains [6] and might also be applicable to MBSE [7]. LLMs demonstrate capabilities to

solve general and domain specific natural and formal language problems [8]. These problems are for example generating text or intelligent search and reasoning, which might be applied in an agentic support set-up. The application of LLMs in the engineering domain has already been demonstrated in the context of requirements engineering [9]. System modeling languages such as SysML, UML or Capella are semi-formal (diagrammatic) languages [10], [11]. This semi-formal class of systems engineering languages differs from the natural languages LLMs have been trained on originally, and can contain complex custom syntax, and even executable [12]. This might pose challenges for the usage of LLMs in this context. Next to that, people with knowledge of its internal structure can analyze LLM-based reasoning with some effort, but it is not easily understood by laypeople. In addition, they may have a tendency to hallucinate [8]. Coincidentally with the maturation and broad application of LLMs, the SysML v1 language is updated targeting an improved machine readability in SysML v2. The sum of these accelerating and limiting factors pose the question of the art of applying LLMs to system modeling tasks and the effect it provides. The following research questions shall be answered:

- 1) In what form are LLMs used for system modeling tasks?
- 2) How is modeling performance evaluated?
- 3) How can future research address current limitations?

In order to answer these questions, this paper first provides a summary of LLM applications related to systems engineering modeling as described in literature (section II). The subsequent section III describes the implementation of a system modeling language translation method, demonstrating the general feasibility of the service and the initial findings. In section IV, implications on future research directions are introduced.

II. OVERVIEW OF THE CURRENT STATE OF RESEARCH

To investigate the field of automated creation, analysis, modification, and reformulation of system models using LLMs,

first, relevant background information and concepts are described. Next, a semi-structured literature review is executed and results are presented.

A. System Modeling Understanding

Based on their experience, the authors identified four tasks that engineers carry out on system models for which LLM support could be beneficial. *Creation* refers to the creation of model artifacts, also referred to as elements such as blocks or ports, or entire models of varying maturity. *Analysis* comprises checking conformance to syntactic and semantic modeling rules, system or technical specification and standards in order to high-light modeling errors or mismatch between modeled elements, diagrams or toward context information such as applied in classical 4-eyes-reviews. *Modification* refers to changes applied to an existing diagram or its elements according to given specific needs (e.g. renewed requirements). *Re-formulating* refers to the translation or transformation between dialects of one system modeling language, multiple languages, or other representations in a way that retains the original model meaning. The LLM application will be allocated to these engineering tasks.

B. Related Approaches

There is already a history of related approaches, using either deterministic methods for automating the creation of (system) models, or other techniques.

This involves earlier approaches for the creation of diagrams using general methods of natural language processing (NLP) [13]–[15]. While this demonstrates the general feasibility, the approaches require a bigger framework with different modules and steps for the generation, and are currently focusing on rather specific models and generation tasks, potentially making the generalization of the approaches for other tasks challenging.

Also, there has been work investigating the transformation of content of diagrams for other, following usage activities, like simulations [16]–[18]. In order to use the transformations, the system needs to be modeled in a predefined way, limiting the scope of application.

One noteworthy contribution discusses the conversion of SysML diagrams from v1 to v2 [19]. While this provides a good overall process for the conversion, covering a wide range of diagrams, the actual process of transformation is not described in detail, and requires manual decisions with semantic understanding of the models.

Overall, the listed related approaches oftentimes provide good preliminary work in the field of creation and reformulation of system models. However, they have challenges with adapting to more complicated models, generalizing for a wider range of models, and, sometimes, lack required semantic understanding that would be needed for this.

C. Techniques to Improve the Output of LLMs

The system modeling domain is characterized by graphical representations and semi-formal languages complemented by

figures, mathematical expressions and numbers. In addition, available training corpora for system models are smaller compared to textual ones [20]. Originated in language processing and trained on huge samples, off-the shelf LLMs might struggle with modeling tasks. They therefore benefit from different mechanisms improving their response quality.

LLMs are executed based on an input called a prompt, which might be given in natural language in our case by an engineer. Prompts presented might be ambiguous, incomplete, over-sized etc., and consequently will the result look like. The discipline of prompt engineering proposes formats and strategies, such as the *chain of thoughts* [21], i.e. split a one-shot prompt in consecutive, less comprehensive tasks. This type of structured and formalized input supports the output quality. Another strategy is the few-shot prompting [21], i.e. a small, well-labeled data set is used in order to adapt the an LLM to the domain need [22]. When applying Retrieval Augmented Generation (RAG) complementary external knowledge is provided in addition to the original prompt, e.g. in form of a graph, [23]. Both methods improve the contextual awareness of the LLM and thereby the output.

In addition to techniques targeting the input for an LLM to achieve better results, fine-tuning slightly adapts the LLM itself without requiring complete retraining of the model [24]. This proves to be helpful in cases where extensive additional domain knowledge, style or existing rules are needed which cannot be comprehensively provided through the input. Next to the larger adaptation hyperparameter or temperature settings frame the LLM's *creativity* or tendency for hallucination [20]. In addition to modifying the LLM itself, rule-based post-processing of the provided output can be leveraged to convert it into a desired form or format [21] or might request dedicated improvements. The different approaches (improving the prompt, the contextual awareness, fine-tuning the model, or post-processing) might be also combined.

The literature review will provide a state of the art landscape of the above mentioned approaches for the realm of system modeling and identify best practices and limitations.

D. Literature Review Approach

The purpose of the review is to answer research questions 1 and 2. To identify works, we considered research from 2022 onward, since LLM technology broadly appears after this time. We used the terms (*LLM or AI*) in combination with the terms (*requirement or system model*) or in combination with one of the engineering related modeling tasks (*understanding, creation, generation, analysis, modification, reformulation, translation, prompting*) and in combination with one of the modeling languages for the search. In addition, (*LLM or AI and system model*) was searched with the terms (*metrics, evaluation or hallucination*). The search was executed in Google Scholar. Contributions are considered, if a peer review exists, and if an implementation is carried out in the realm of diagrammatic creation, analysis, modification, or reformulation. From the corpus of search results, 20 papers were deemed to be relevant. The relevance was decided by

independent assessment of two raters, who are authors of this paper. For cases of disagreement, further authors discussed and mutually agreed on the case.

E. Results of the Review

The results of the review are summarized in I. It provides an overview by assorting the sources to the modeling tasks, potential improvements strategies on the LLM and evaluations carried out with respect to the output. Papers addressing *modification* or *analysis* also contribute to *creation*, they are mentioned only once in the table in the most relevant category.

The improvement category summarizes modification or addition to the LLM application in the four categories *Few-Shot Prompting*, *RAG*, *Fine-Tuning* and *Post-Processing*. Independent from the improvement strategy contributions might use simple one shot prompts, rather sophisticated pattern or split task approaches. The category *Few-Shot* contains papers mentioning modeling language pattern or any other structured context, the category *RAG* assorts all papers naming RAG directly or provide ontological structures that serve as generic input to the LLM. The category *Post-Processing* summarizes papers making use of deterministic quality checks, feedback loops or format changes. Contributions might use combinations of the technologies (e.g. RAG and post-processing) and therefore appear several times. Six papers ([25]–[30]) out of the twenty (i.e. %30) did not use any of these improvement strategies.

Evaluation strategies are grouped in *Qualitative*, *Comparison* and *Automatic* in I. Papers listed in the *Qualitative* column completed a questionnaire judging the result for dedicated features or capabilities from a user perspective. For papers listed in the column *Comparison* a manually created diagram is compared to an LLM created one. Also in this category, a questionnaire for dedicated features or categories might be answered. Some authors, e.g. [34], also use automated measures such as calculated distances to quantify the difference. The third category *Automatic* refers to LLM specific KPIs such as ROUGE [43] or other Machine Learning related KPIs such as F1 score, precision, recall or computational cost etc. [44].

The purpose of the LLM usage was summarized in two categories, the *quality* of a model or the overall *efficiency* of modeling tasks, which are allocated to the *Value Proposition* in 1. Exemplary application cases might be the effort required to generate architectural breakdowns based on specification and requirements in the category *Efficiency* or the search for noncompliance in functional models that have been created by students in the category *Quality*.

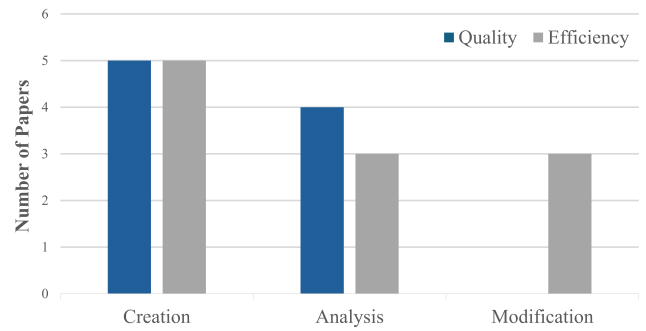


Figure 1. Overview of the Value Proposition per Modeling Task

It was observed, that most papers refer to SysML v1 (8), SysML v2 (3) or UML (5). One exception is [34], which builds on the OPM language, as well as [35], which proposes solutions for Capella. Many authors report serializing into XMI or JSON format before or after the interaction with the LLM. System models appear in different diagram types. 2 gives an overview of the type of diagram the twenty papers dealt with.

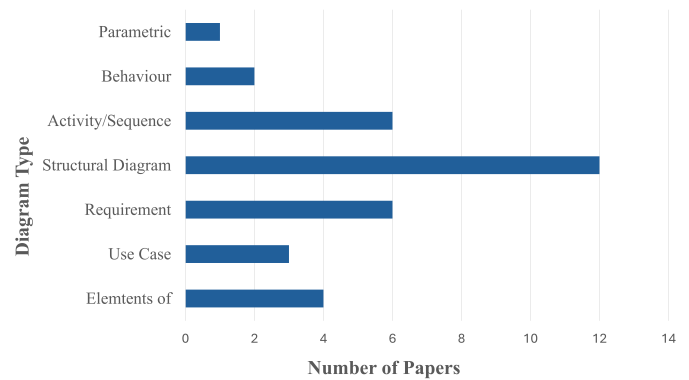


Figure 2. Overview of Diagram Types

One Paper [45] works explicitly on diagram elements only, [32], [25] and [24] handle both diagrams and elements, which fall in the category *Elements of* in 2. Many authors describe several diagram types in their contributions, whereas [37] does not detail which diagram type was subject to the application.

Most proposed solutions establish either new co-piloted modeling environments or work in two separated environments - LLM user interface and conventional modeling tool. This is often accompanied by intermediate steps in development

Table I
OVERVIEW OF LITERATURE REVIEW RESULTS

Application	Total	Improvement-Strategy				Evaluation		
		<i>Few-Shot</i>	<i>RAG</i>	<i>Fine-Tuning</i>	<i>Post-Processing</i>	<i>Qualitative</i>	<i>Comparison</i>	<i>Automatic</i>
Creation	10	[5] [34]	[31], [5]	[32], [33]	[31], [32], [33] [35]	[32], [30], [5], [33]	[31], [28], [29] [34], [35], [27]	[35]
Analysis	7	[36], [37]	[38]	[39], [40]	[36], [37], [39] [40], [41]	[36], [37] [40], [41]	[39]	[38], [26] [41]
Modification	3	[24], [42]	[24]			[24], [42], [25]		[25]

environments, e.g. in order to transform the system model format. [46] and [35] created an "LLM-plugin" or "co-pilot" functionality in a pre-existing modeling tool connected via an API to a modeling environment.

Within the analyzed literature some authors test and report dedicated improvements based on their applied improvement strategies. [31] describes that post-processing improves semantic errors in the models. [34] reports on the reduction of hallucination, when using few-shot prompting. An improved consistency between different diagrams was found by [36] when applying a combination from few-shot prompting and post processing. [41] finds improved correctness of trace links with post-processing only. [37] finds with the combination of post-processing and few-shot prompting a higher level of model correctness. According to [38], the quality of the LLM's analysis of diagrams improves when RAG is engaged. Many authors do not directly report on the effects of individual improvement strategies since they focus on other objects. Also, the effort involved in implementation is often not reported on, compared to the savings evaluated for the engineer working on the diagrams.

For the evaluation of the LLMs output twelve papers out of the twenty relay on individual case related qualitative questionnaires, asking users e.g. for the perceived correctness, first-time right modeling, completeness of individual diagrams, consistency between diagrams, etc, see column *Qualitative* in I. Only five out of twenty use *Quantitative* measures for their evaluation. This conventional methods originating in computer science lack adaptation to the engineering context. This means, that measures originated in systems engineering, such as for example the correctness or completeness of trace links between diagrams cannot be addressed. The idea of comparing manual with LLM work was mostly used for the creation of system models. Comparisons might give valuable insight also for the other modeling tasks, but require the manual creation of diagrams for evaluation purpose only.

The quality of the LLM service is perceived differently. When working with students or system modeling beginners, authors report positive, ([38], [27]) or conclude on efficiency gains even on a professional level [39]. Other authors remain indifferent, as e.g. [47], with respect to the overall usefulness, and some remain in doubt when working with experienced professionals [29]. This observation leads to the question of reliable research evaluation approaches. Several authors, such as [35] or [27], underpin the need for evaluation approaches.

F. Interim Conclusion

The review showed that fourteen out of twenty authors enhance the off-the-shelf LLM capability by providing context, fine-tuning it, adding post-processing or a combination of these. In addition, prompting strategies and tool embeddings complete the functionality. This answers to research question one. However, the reviewed literature does not provide a complete picture of which improvement strategy best fits potential model weaknesses, nor does it explain how the implementation effort relates to expected benefits.

Evaluation methods are scattered, which limits comparability and reuse of the proposed application. Standard machine learning and also LLM based evaluation have not been built for the engineering application context, therefore there is a lack of quantitative system modeling related evaluation approaches. In addition, LLM based support and service can produce content quicker compared to humans. Thus the main question related to evaluation of model services is on the provided quality as well as the effort needed for creation compared to the expected benefits for users. Further research needs to detail evaluation approaches, standardize them for system modeling purposes and their application to system modeling tasks and dedicated value propositions. This answers to research question two. The V describes the details for future research.

Limitations of the application of LLMs for system modeling tasks mentioned by several authors (e.g. [31], [34]) is the prompting window size, especially with tokenizing effort when working with XMI as intermediate intake.

The engineering task to transform or translate models was not found within the literature review. [39] describes the transformation of requirements into other forms briefly along many other subjects. [24] roughly introduces the opportunity to translate SysMLv1 to SysML v2 based on the v2 language description, but does not implement it. Related approaches showed limitations in generalizing and context understanding. Therefore, the III proposes a translation approach.

III. DESCRIPTION OF AN LLM-BASED SYSML TRANSLATION APPROACH

A. SysML v1 to SysML v2 Translation for Structure Diagrams

With the upcoming release of SysML v2, companies are faced with the task to convert and transition a large number of existing SysML v1 system models of various size and complexity to match the new notation. Unlike SysML v1, SysML v2 does not use UML as semantic baseline but introduces a new core language called KerML. Different semantics describing new mechanics and the redefinition or introduction of new elements greatly complicate the translation task, as there are no general one-to-one equivalences between SysML v1 and v2. The Object Management Group (OMG) provides a translation specification consisting of possible mappings between elements, which however lacks a systematic methodology [48]. Currently, there is no automated transition tool, and every element, diagram, and model needs to be remodeled by hand. Consequently, this section explores the use of LLMs to translate SysML v1 to SysML v2 diagrams by providing a prompt strategy.

The proposed approach focuses on the translation of block definition (BDD) and internal block diagrams (IBD) due to their relative simple semantic structure and concise number of element types. The following approach has been developed and tested in three phases: first, a pre-study analyzing different input and out formats has been carried out. Second, recurring fault types (syntax, semantics), have been analyzed for IBD as well as BDD and distilled into modules. Third, the structure and the module interplay has been tested and iteratively refined

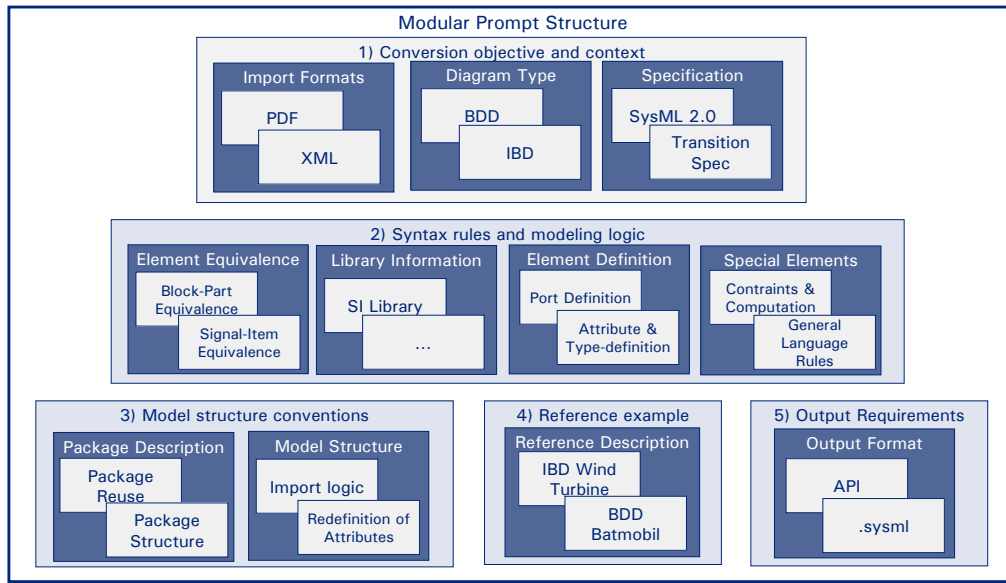


Figure 3. Schema Visualizing the Proposed Modular Prompt Strategy

using various structural diagrams from different systems such as wind turbine, vehicle climate control and safe door opening system. An internal testing concludes on the effectivity of the approach.

B. Modular Prompt Strategy

To reduce the prompting effort and increase the result accuracy for the translation of every diagram, a modular prompt strategy is proposed. By defining standardized prompt blocks and providing usage instructions, the modular prompting approach reduces the translation effort to a module selection process. The core prompt structure consists of the following modules: (1) conversion objective and context, (2) syntax rules and modeling logic, (3) model structure conventions, (4) reference example and (5) output requirements.

The *conversion objective and context* block defines the overarching goal of the translation prompt. It describes what type of SysML diagram is provided in which format as input, and which additional resources in the form of language or conversion specification, as provided by the OMG [48] should be used.

The *syntax rules and modeling logic* block is the largest and most flexible section of the prompt, as it defines how the different elements should be translated syntactically. First, the basic transformation principle is described for the selected diagram type. Here, basic equivalences between all elements present in the diagram type are described. In case of a block definition diagram for example, the equivalence of block to part or signal to item flow are introduced. In addition, the use of specific SysML v2 libraries (e.g. System of Units (SI)) can be specified. Second, general language rules are defined in order to limit syntax errors resulting from semantic discrepancies between both languages. This includes the usage of units, abbreviations and KerML specific prepositions such

as "in" and "out" for flow elements. Third, a specific block describing the conversion and equivalence is added for every element type present in the diagram. As SysML v2 defines features and their inheritance differently than SysML v1, inheritance and generalization mechanisms are described.

The *model structure convention* block addresses the core semantic structure of the diagram. The introduction of views and packages as well as libraries in SysML v2 requires an understanding of the underlying package-structure and import logic in order to limit or eliminate the local redefinition of elements for every diagram. SysML v2 also introduces "private", "protected" and "public" specifications with no direct equivalent in SysML v1. Consequently, especially cross-package reuse of elements requires explicit prompting.

The *reference example* block specifies which SysML v2 example model should be used as translation reference. Especially larger models with multiple element types and a specific naming convention across multiple elements, are hard to translate. The LLM benefits from an example as it clarifies potential semantic conflicts. The word "fuel" for example might get used as part "fuel tank" as port "fuel outlet" or as item "fuel".

The *output requirements* block describes the desired output format such as a .SysML file or associated SysML v2 API calls necessary to generate the new SysML v2 model.

3 summarizes the modular prompt approach. Each module consists of either a diagram or element specific prompt section that needs to be selected according to the translation goal. Each module is then situated or referenced in the prompt strategy. Currently, the prompt structure and the modules have been developed, implemented and tested for structure diagrams in particular BDD and IBD.

C. Internal Testing

10 different diagrams from 4 different models (Safe Door Opening System, Wind Turbine, Vehicle Climate Control, Coffee Machine) have been used to test the modular prompt for both BDD and IBD. Diagram sizes ranging from 6 to 40 elements each encompassing different combinations of element-types (e.g. block, interface, signal, proxy, port) have been selected. Five users from the department tested the prompt system. For the test gemini 2.5 flash online tool and chat gpt models 4o, 4.5, o1 and o3 mini have been used. The diagrams are provided as .PNG file and contain five to ten blocks with four to ten ports each. Linkage between the elements contains energy provision, the flow of the produced goods, signals for machine control, conversation with users and data exchange with other software. For improving the quality further existing knowledge could be provided e.g. as RAG containing given SysML v1 to v2 transformation guidelines.

D. Preliminary Summary of Prompt Strategy

The proposed modular prompt strategy aims to support the transformation of SysML v1 to v2 diagrams by providing a systematic prompt description. This reduces the prompting effort of LLMs and can therefore increase the efficiency of LLM-based support tools for SysML modeling. An industrial application of prompts like the one proposed, would require highly disciplined methodological work. Therefore, embedding of the consecutive prompting steps in a user interface allowing for automation or a workflow sequence is recommended when industrializing. Furthermore, ensuring trustworthiness reliable, multifaceted, yet precise evaluation metrics and benchmarks are needed to assess the modeling performance of LLMs.

IV. FUTURE IMPLICATIONS

The previous sections showed the great potential of supporting efficient and qualitatively high system modeling support with LLMs, especially when enhanced with domain knowledge. However, context provision or adaptation to domain context requires significant effort. In parallel, the development speed of LLMs and related technology is considered to remain high. This means that future LLM systems will feature modular or adjustable architectures, enabling rapid adaptation — as exemplified by the newly introduced Model Context Protocol (MCP), [49]. This section sketches a possible way of providing extended engineering context allowing also for more complicated models thereby responds to research question three.

Co-piloting services will be successful if they provide high-quality system models reliably in a comparably short duration, i.e. if they relieve the engineer from modeling effort without requiring excessive manual postprocessing. Alternatively, co-piloting must provide high-quality addition or correction services for specific segments or elements. Investigation is required to assess to what extend technical improvements on pretrained LLMs, as discussed in II, are sufficient for this type of high-quality co-piloting and to what extend small

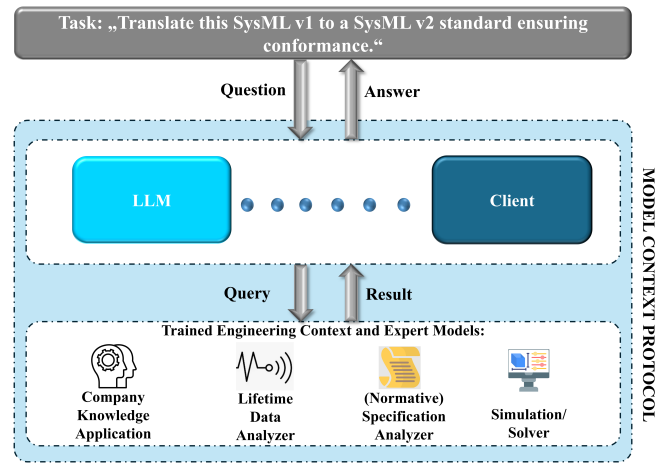


Figure 4. MCP-based Expert Decision for a SysML Translation Service

language models or sophisticated prompting strategies fulfill the users' need of a reliable and trustful output. Independent from engineering decisions, it is expected that future LLMs feature more modalities, that will be supportive in handling the high variety of engineering data, e.g. normative or other specification and documentation comprising pictures, tables and text at the same time, various simulation and geometrical models, knowledge in databases, graph structures or formulated as heuristics, expert or decision systems, life-time data and so on. This will provide much more context than prompts currently provide and is expected to offer improved content and freer input. However, teaching LLMs in any engineering model type, data class and data format might not be beneficial because of their variety. Therefore, linking existing, trained context or expert models or existing rule based applications to a broader AI system might support higher informed reasoning. The MCP client server structure, as proposed by [49], is one possible solution for a broader AI system. For the translation case it might be applied as depicted exemplarily in 4.

The LLM analyses the user's request and selects required services or contextual required information. The client requests these services via query in the linked applications or pre-trained decision support. For the SysML translation service this could be *company knowledge* related to the component or system of interest described as well as SysML v1 to v2 translation guides in the *specification analyzer*. This architecture would release the LLM input from extensive prompts including context information and forcing a dedicated structure.

The parallel advent of SysML v2 introduces a textual notation, APIs and improved direct machine readability and therefore represents high potential for an easier fit to LLM technologies. As different diagrams are replaced by different views of a single metamodel similar file, a broader, deeper, and more complex requirement-structure-behavior relationship networks can be described centrally, opening the door for the creation of diagram transcending, complete system represent-

ing, models, generated by a LLM co-pilot. In addition, the new standard introduces element "definitions" and "usages" in an occurrence-driven modeling approach. This enables new ways of modeling time dependent variants of systems as well as a systematic description of the trace between structure and behavior of a system. The resulting increased model complexity further drives the need for modeling co-pilots.

With the further evolving AI systems it is imperative to establish guidelines for engineering decision making including security and safety considerations.

V. CONCLUSIONS AND FUTURE RESEARCH

This paper analyzed the current state of application of LLMs for the systems modeling. It was found that LLMs, if improved with respect to the engineering context and tasks, are perceived useful. A literature review showed that evaluation approaches are scattered and comparability between results is of general research interest. A transformation approach based on prompts has been introduced for SysML v1 and SysML v2. To overcome the reported lack of input size and need for context awareness when tasked with system modeling, an MCP approach is proposed. It remains open, how much improvement in terms of system model quality, efficiency and freedom in the interaction can be achieved. A detailed evaluation for the translation approach as well as an implementation and evaluation to the MCP approach are subject to further research.

The above sections showed that different evaluation approaches are used in a way that limits reproducibility and comparability, see also [34]. The resulting question is: how to ensure rigor when proposing AI based modeling support? One way to start could be to define commonly agreed evaluation criteria matching the engineering tasks and to provide standardized assessment methods. However, evaluating system models is not straightforward in itself, as [50] points out. That means, an easy definition of a set of KPIs for "good" models is not available. The literature review showed partial use of comparative approaches requiring a manually created comparison diagram to be at hand. Due to the effort and individualized approach, the authors propose to neglect the manual comparison. Computer scientists developed models such as ROUGE [43] for summary quality or BLEU or BERT for translation quality. These are classifiers use a set of manually created and labeled comparison data, such as summaries of a text. These models may be used to automatically determine the output quality for dedicated purposes. The extent to which system modeling tasks can be formulated in a generalized manner so that automated quality evaluation becomes possible remains open. Therefore, automatically calculated measures indicating e.g. the accuracy, might be combined with qualitative evaluation tools on the user's perception.

REFERENCES

- [1] M. Schöck, J. Müller, M. Batora, and A. Albers, "A comprehensive approach for the strategic determination of individual target levels of agility in product development," in *DS 130: Proceedings of NordDesign 2024, Reykjavik, Iceland, 12th–14th August 2024*, J. Malmqvist, M. Candi, R. J. Sæmundsson, F. Byström, and O. Isaksson, Eds. Reykjavik, Iceland: The Design Society, 2024, pp. 711–719.
- [2] L. Zirui, F. Faheem, and S. Husung, "Collaborative model-based systems engineering using dataspace and sysml v2," *Systems*, 2024.
- [3] L. Zhang and Z. Chen, "Mbse 2.0: A more integrated, comprehensive, and intelligent mbse," in *Comprehensive, and Intelligent MBSE*. SSRN, 2024.
- [4] M. Alenazi, N. Niu, and J. Savolainen, "Sysml modeling mistakes and their impacts on requirements," in *2019 IEEE 27th International Requirements Engineering Conference Workshops (REW)*. IEEE, 2019, pp. 14–23.
- [5] Q. Xu, L. Zhang, T. Li, and X. Wang, "Llm-enhanced sysml formal method for flanking protection in train autonomous control system," in *International Conference on Artificial Intelligence and Autonomous Transportation*. Springer, 2024, pp. 459–471.
- [6] L. Regenwetter, A. H. Nobari, and F. Ahmend, "Deep generative models in engineering design: a review," *Journal of Mechanical Design*, 2022.
- [7] D. Tissen, R. Bernijazov, C. Koldewey, and R. Dumitrescu, "A maturity model for data-driven model-based systems engineering for producing companies," in *DS 134: Proceedings of the 26th International DSM Conference (DSM 2024)*, Stuttgart, Germany. The Design Society, 2024.
- [8] H. Zhao, H. Chen, F. Yang, N. Liu, H. Deng, H. Cai, S. Wang, D. Yin, and M. Du, "Explainability for large language models: A survey," *ACM Transactions on Intelligent Systems and Technology*, vol. 15, no. 2, 2024. [Online]. Available: <https://doi.org/10.1145/3639372>
- [9] C. Ellsel and R. Stark, "Advancing Requirements Engineering with Large Language Models," in *Procedia CIRP 136C (2025)*, Aug. 2025, pp. 701–706.
- [10] P. D. SAQUI-SANNES, R. A. VINGERHOEDS, C. GARION, and X. THIRIOUX, "A taxonomy of mbse approaches by languages, tools and methods," *IEEE Access*, 2022.
- [11] L. Timperley, C. Snider, and T. Tryfonas, "Mapping the mbse environment and complementary design space exploration techniques," in *2024 IEEE Aerospace Conference*, 2024.
- [12] C. Ellsel, G. Senkal, and R. Stark, "Using MBSE With SysML to Analyze Change Impacts, Including Efforts, in Technical Systems," in *2022 IEEE International Symposium on Systems Engineering (ISSE)*. Vienna, Austria: IEEE, Oct. 2022, pp. 1–8. [Online]. Available: <https://ieeexplore.ieee.org/document/10005450/>
- [13] D. K. Deeptimahanti and M. A. Babar, "An Automated Tool for Generating UML Models from Natural Language Requirements," in *2009 IEEE/ACM International Conference on Automated Software Engineering*. Auckland, New Zealand: IEEE, Nov. 2009, pp. 680–682. [Online]. Available: <http://ieeexplore.ieee.org/document/5431706/>
- [14] D. K. Deeptimahanti and R. Sanyal, "Semi-automatic generation of UML models from natural language requirements," in *Proceedings of the 4th India Software Engineering Conference*. Thiruvananthapuram Kerala India: ACM, Feb. 2011, pp. 165–174. [Online]. Available: <https://dl.acm.org/doi/10.1145/1953355.1953378>
- [15] C. R. Narawita and K. Vidanage, "UML generator - an automated system for model driven development," in *2016 Sixteenth International Conference on Advances in ICT for Emerging Regions (ICTer)*. Negombo, Sri Lanka: IEEE, Sep. 2016, pp. 250–256. [Online]. Available: <http://ieeexplore.ieee.org/document/7829928/>
- [16] V. H. Ngo and T. Soriano, "A model transformation process to realize controllers of ship autopilot systems by the specialized MDA's features with UML/SysML," in *2012 9th France-Japan & 7th Europe-Asia Congress on Mechatronics (MECATRONICS) / 13th Int'l Workshop on Research and Education in Mechatronics (REM)*. Paris, France: IEEE, Nov. 2012, pp. 20–26. [Online]. Available: <http://ieeexplore.ieee.org/document/6450983/>
- [17] G.-D. Kapos, A. Tsadimas, C. Kotronis, V. Dalakas, M. Nikolaidou, and D. Anagnostopoulos, "A Declarative Approach for Transforming SysML Models to Executable Simulation Models," *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, vol. 51, no. 6, pp. 3330–3345, Jun. 2021. [Online]. Available: <https://ieeexplore.ieee.org/document/8752097/>
- [18] S. Salunkhe, R. Berglehner, and A. Rasheeq, "Automatic Transformation of SysML Model to Event-B Model for Railway CCS Application," in *Rigorous State-Based Methods*, A. Raschke and D. Méry, Eds. Cham: Springer International Publishing, 2021, vol. 12709, pp. 143–149, series Title: Lecture Notes in Computer Science.

- [19] "Technical Report: SysML v1 to SysML v2 Model Conversion Approach," Object Management Group, Tech. Rep., 2024. [Online]. Available: <https://www.cto.mil/wp-content/uploads/2025/02/SysML-v2-TransitionApproach-1.3.pdf>
- [20] R. Chen, J. Shen, and X. He, "A model is not built by a single prompt: Llm-based domain modeling with question decomposition," in *n.a. n.a.*, 2024.
- [21] J. D. Rocco, D. DiRuscio, C. D. Sipio, P. T. Nguyen, and R. Rube, "On the use of large language models in model-driven engineering," *Software and Systems Modeling* (2025) 24, p. 923–948, 2024.
- [22] X. Li, S. Yuan, X. Gu, Y. Chen, and B. Shen, "Few-shot code translation via task-adapted prompt learning," *Journal of Systems and Software*, 2024.
- [23] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge intensive nlp tasks," in *Proceedings of the 34th International Conference on Neural Information Processing Systems, NIPS*, 2020.
- [24] R. Longshore, R. Bell, and R. Madachy, "Leveraging generative ai to modify and query mbse models," in *Proceedings of the Twenty-First Annual Acquisition Research Symposium*. ARP, 2024.
- [25] S. Saloranta, "From documents to models: Adopting mbse practices in complex embedded systems and leveraging large language models," Master's thesis, University of Turku, 2024.
- [26] J. Hassine, "Evaluating multi-modal llms for automatically recognizing semantic elements in uml use case diagram images," in *2025 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 2025, pp. 861–866.
- [27] B. Wang, C. Wang, P. Liang, B. Li, and C. Zeng, "How llms aid in uml modeling: an exploratory study with novice analysts," in *2024 IEEE International Conference on Software Services Engineering (SSE)*. IEEE, 2024, pp. 249–257.
- [28] D. De Bari, G. Garaccione, R. Coppola, M. Torchiano, and L. Ardito, "Evaluating large language models in exercises of uml class diagram modeling," in *Proceedings of the 18th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement*, ser. ESEM '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 393–399.
- [29] B. Johns, K. Carroll, C. Medina, R. Lewark, and J. Walliser, "AI Systems Modeling Enhancer (AI-SME): Initial Investigations into a ChatGPT-enabled MBSE Modeling Assistant," *INCOSE International Symposium*, vol. 34, no. 1, pp. 1149–1168, Jul. 2024. [Online]. Available: <https://incose.onlinelibrary.wiley.com/doi/10.1002/iis2.13201>
- [30] A. Ferrari, S. Abualhaijal, and C. Arora, "Model generation with llms: From requirements to uml sequence diagrams," in *2024 IEEE 32nd International Requirements Engineering Conference Workshops (REW)*. IEEE, 2024, pp. 291–300.
- [31] L. Aprville and B. Sultan, "System architects are not alone anymore: Automatic system modeling with ai," in *MODELSWARD 2024: 12th International Conference on Model-Based Software and Systems Engineering*, vol. 1. SCITEPRESS-Science and Technology Publications, 2024, pp. 27–38.
- [32] P. J. Kulkarni, D. Tissen, R. Bernijazov, R. Dumitrescu *et al.*, "Towards automated design: Automatically generating modeling elements with prompt engineering and generative artificial intelligence," *DS 130: Proceedings of NordDesign 2024, Reykjavik, Iceland, 12th-14th August 2024*, pp. 617–625, 2024.
- [33] C. Paniagua and F. L. Caso, "Models2code: Autonomous model-based generation to expedite the engineering process," *Systems Engineering*, vol. 28, no. 2, pp. 224–237, 2025.
- [34] R. M. G. Alarcia, P. Russo, A. Renga, A. Golkar *et al.*, "Bringing systems engineering models to large language models: An integration of opm with an llm for design assistants," in *MODELSWARD*, 2024, pp. 334–345.
- [35] L. R. Timperley, L. Berthoud, C. Snider, and T. Tryfonas, "Assessment of large language models for use in generative design of model based spacecraft system architectures," *Journal of Engineering Design*, pp. 1–21, 2025.
- [36] B. Sultan and L. Aprville, "Ai-driven consistency of sysml diagrams," in *Proceedings of the ACM/IEEE 27th International Conference on Model Driven Engineering Languages and Systems*, ser. MODELS '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 149–159.
- [37] H. Kausch, K. Koppes, L. Netz, P. O'Brien, M. Pfeiffer, D. Raco, M. Radny, A. Rath, R. Richstein, and B. Rumpe, "Applied model-based co-development for zero-emission flight systems based on sysml," in *Proceedings of the Deutscher Luft und Raumfahrt Kongress. Die Deutsche Gesellschaft für Luft-und Raumfahrt (DGLR) e. V. Lilienthal ...*, 2024, p. 10.
- [38] P. Ardimento, M. L. Bernardi, and M. Cimitile, "Teaching uml using a rag-based llm," in *2024 International Joint Conference on Neural Networks (IJCNN)*. IEEE, 2024, pp. 1–8.
- [39] J. Bourdon, J. Rodriguez, Q. Lesigne, P. Suchet, B. Fister, L. Montagne, O. Malhomme, L. Benmiloud-Bechet, and R. Plana, "Transforming systems engineering in nuclear projects with generative ai: A path to efficiency and compliance," in *Proceedings of the 13th International Conference on Model-Based Software and Systems Engineering*, 2024.
- [40] E. Bader, D. Vereno, and C. Neureiter, "Facilitating User-Centric Model-Based Systems Engineering Using Generative AI:," in *Proceedings of the 12th International Conference on Model-Based Software and Systems Engineering*. Rome, Italy: SCITEPRESS - Science and Technology Publications, 2024, pp. 371–377.
- [41] P. Darm, F. Marchetti, G. Garcia, P. M. Redondo, A. Riccardi, and A. G. Fernández, "Leveraging language models semantic similarity capabilities to facilitate information reuse in system engineering," in *In: Proceedings of 74th International Astronautical Congress*, 2023.
- [42] J. K. DeHart, "Leveraging Large Language Models for Direct Interaction with SysML v2," *INCOSE International Symposium*, vol. 34, no. 1, pp. 2168–2185, Jul. 2024. [Online]. Available: <https://incose.onlinelibrary.wiley.com/doi/10.1002/iis2.13262>
- [43] C.-Y. Lin, "ROUGE: A package for automatic evaluation of summaries," in *Text Summarization Branches Out*. Barcelona, Spain: Association for Computational Linguistics, Jul. 2004, pp. 74–81. [Online]. Available: <https://aclanthology.org/W04-1013/>
- [44] G. Naidu, T. Zuva, and E. M. Sibanda, "A review of evaluation metrics in machine learning algorithms," in *Computer science on-line conference*. Springer, 2023, pp. 15–25.
- [45] J. K. DeHart, "Leveraging large language models for direct interaction with sysml v2," in *INCOSE International Symposium*, vol. 34, no. 1. Wiley Online Library, 2024, pp. 2168–2185.
- [46] S. Saloranta, "From Documents to Models: Adopting MBSE Practices in Complex Embedded Systems and Leveraging Large Language Models," Master's thesis, University of Turku, 2024.
- [47] D. De Bari, "Evaluating Large Language Models in Software Design: A Comparative Analysis of UML Class Diagram generation," Master's thesis, Politecnico Torino, 2024.
- [48] *OMG Systems Modeling Language™ (SysML®) Part 2: SysML v1 to SysML v2 Transformation*, February 2024. [Online]. Available: <https://www.omg.org/spec/SysML/2.0/Beta2/About-SysML>
- [49] *News: Introducing the Model Context Protocol*, Anthropic, 2025.
- [50] D. L. Moody, "Theoretical and practical issues in evaluating the quality of conceptual models: Current state and future directions," in *Data and Knowledge Engineering*, 55(3), 2005, p. 243–276.